High Performance Computing

GPU Programming with CUDA

Ayazbek Abdanur, BDA2406

Using Google Colab

A local NVIDIA GPU is **not required** for this exercise. All CUDA programs can be compiled and executed using a GPU provided through Google Colab.

Step 1: Create a Colab Notebook

Open Google Colab in your web browser and create a new notebook.

Save the notebook so that your work is not lost if the runtime disconnects.

Step 2: Enable a GPU

In the Colab menu select:

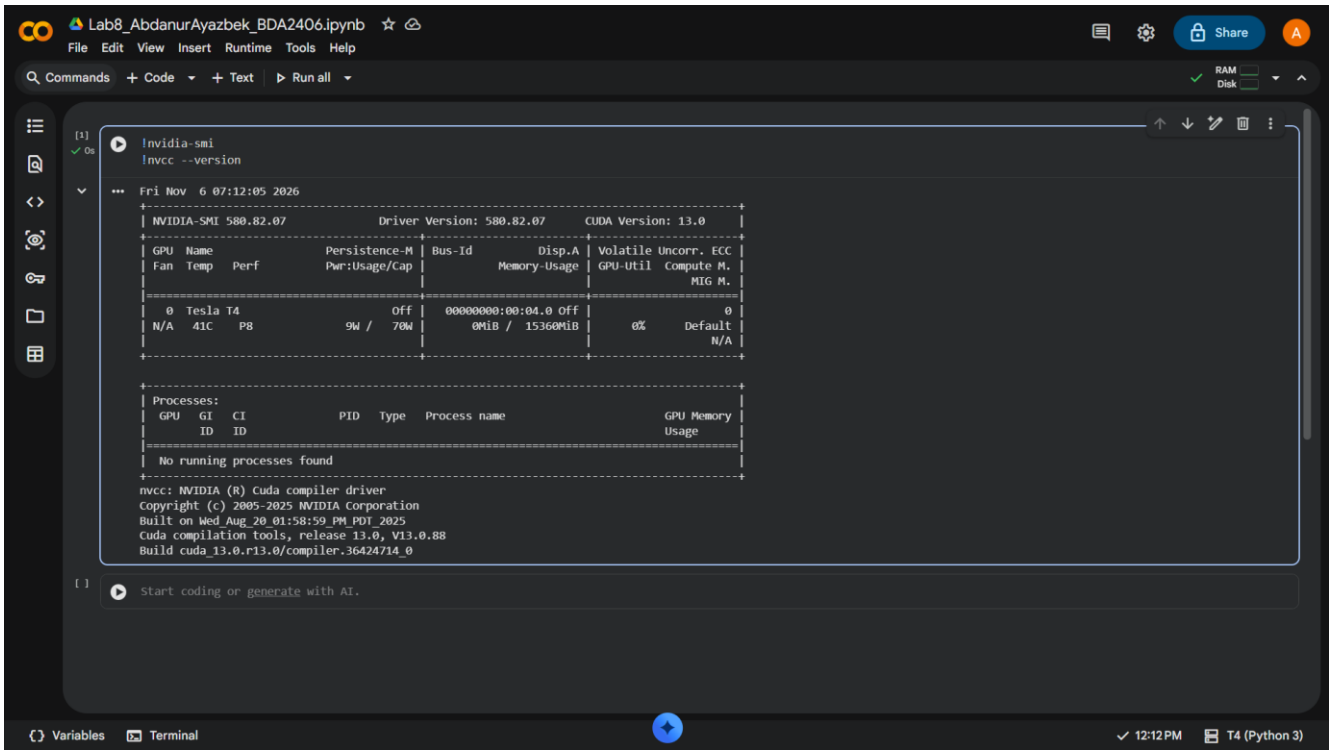
Runtime → Change runtime type → Hardware accelerator → GPU

Connect to the runtime.

GPU availability depends on Colab resources. The exact GPU model assigned to your notebook may be different for different students.

Step 3: Check the GPU

Create a code cell and run:



```
!nvidia-smi
```

You should see information about the NVIDIA GPU assigned to your runtime.
Record the GPU model:

Item	Result
GPU model	Tesla T4
GPU memory	15360 MiB (about 15 GB)

Step 4: Check the CUDA Compiler

Run:

```
!nvcc --version
```

The nvcc command is the NVIDIA CUDA compiler.
Record the CUDA version reported by the system.

Item	Result
CUDA version	13.0

Creating CUDA Files in Colab

CUDA programs normally use the extension:

.cu

You can create a CUDA source file directly from a Colab cell using:

%%writefile program.cu

For example:

```
%%writefile hello.cu

#include <stdio.h>

int main()
{
    printf("Hello\n");
    return 0;
}
```

Then compile the file in another cell:

```
!nvcc hello.cu -o hello
```

and execute it:

```
!./hello
```

Throughout this exercise, use the same procedure:

1. create a .cu source file;
2. compile it using nvcc;
3. execute the compiled program;
4. record the requested results.

Objectives

The purpose of this exercise is to become familiar with the basic CUDA programming model and GPU programming. You will distinguish between host and device execution, create and launch CUDA kernels, work with blocks and threads, calculate global thread indices, allocate GPU memory, transfer data between CPU and GPU memory, perform vector operations in parallel, and investigate how the execution configuration affects a CUDA program.

Part 1: Your First CUDA Kernel

Create the file:

hello.cu

with the following program:

```
# include      <stdio .h>
#include <cuda_runtime.h>

__global__ void helloKernel()
{
    printf("Hello from the GPU!\n");
}

int main()
{
    printf("Hello from the CPU!\n");

    helloKernel <<<1,  1>>>();

    cudaDeviceSynchronize();

    return 0;
}
```

Compile:

```
!nvcc hello.cu -o hello
```

Run:

```
!./hello
```

Answer the following questions.

1. Which function executes on the CPU?

main(). It prints "Hello from the CPU!" and launches the kernel

2. Which function executes on the GPU?

helloKernel(). It prints "Hello from the GPU!"

3. What does the __global__ qualifier mean?

The function runs on the GPU and is called from the CPU code

4. What is a CUDA kernel?

A function that runs on the GPU and is marked with __global__. When it is launched, many GPU threads can run the same function in parallel

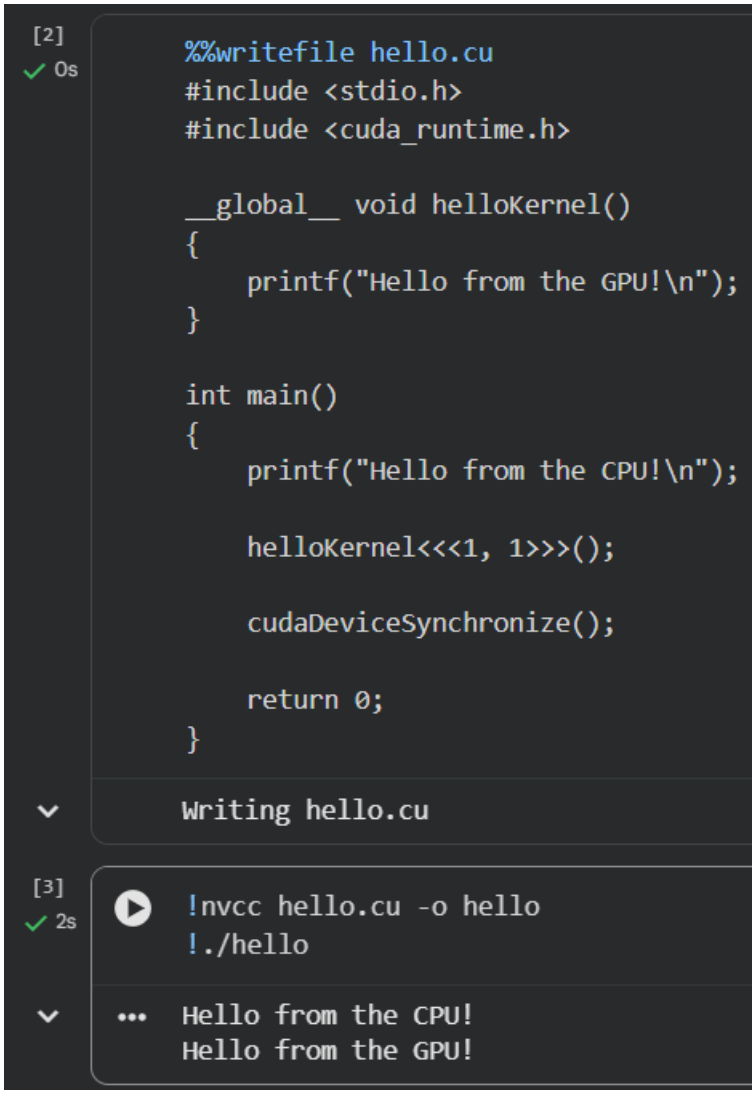
5. What does the following syntax represent?

```
helloKernel<<<1, 1>>>();
```

A kernel launch. The CPU tells the GPU to run helloKernel with the configuration written inside <<< >>>

6. What do the two values inside <<<1,1>>> specify?

The first value is the number of blocks, the second value is the number of threads in each block. Here it is 1 block with 1 thread, so the kernel runs only once



Part 2: Blocks and Threads

Modify the kernel:

```
__global__ void helloKernel()
{
    printf("Block %d, Thread %d\n",
           blockIdx.x,
           threadIdx.x);
}
```

Launch the kernel using:

```
helloKernel<<<2, 4>>>();
```

Run the program and complete the table.

[4] ✓ 0s

```
%%writefile blocks_threads.cu
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void helloKernel()
{
    printf("Block %d, Thread %d\n",
           blockIdx.x,
           threadIdx.x);
}

int main()
{
    printf("Hello from the CPU!\n");

    helloKernel<<<2, 4>>>();

    cudaDeviceSynchronize();

    return 0;
}
```

```
Writing blocks_threads.cu
```

[5] ✓ 1s

```
!nvcc blocks_threads.cu -o blocks_threads
!./blocks_threads
```

```
Hello from the CPU!
Block 0, Thread 0
Block 0, Thread 1
Block 0, Thread 2
Block 0, Thread 3
Block 1, Thread 0
Block 1, Thread 1
Block 1, Thread 2
Block 1, Thread 3
```

Block	Possible Thread IDs
0	0, 1, 2, 3
1	0, 1, 2, 3

Answer:

1. How many blocks are created?

2 blocks. The first value in <<<2, 4>>> is the number of blocks

2. How many threads are created inside each block?

4 threads. The second value in <<<2, 4>>> is the number of threads in each block

3. How many threads are created in total?

8 threads, because 2 blocks × 4 threads = 8

4. What does blockIdx.x represent?

The number of the block that the thread belongs to. It starts from 0, so here it is 0 or 1

5. What does threadIdx.x represent?

The number of the thread inside its block. It starts from 0 in every block, so here it is 0, 1, 2 or 3

6. Is the order in which the messages are printed guaranteed?

No. Blocks and threads run in parallel, and the GPU decides which block runs first. The output can come in a different order in different runs, even if in this run block 0 was printed first

Part 3: Global Thread Index

Thread numbers repeat inside every block.

For example, with:

<<<3, 4>>>

every block contains threads numbered:

$$0,\quad 1,\quad 2,\quad 3.$$

Therefore, a global thread index can be calculated as:

$$\text{index} = \text{threadIdx.x} + \text{blockIdx.x} \times \text{blockDim.x}.$$

Modify the kernel:

```
__global__ void showIndex()
{
    int index =
        threadIdx.x +
        blockIdx.x * blockDim.x;

    printf("Block %d, Thread %d, Global index %d\n",
           blockIdx.x,
           threadIdx.x,
           index);
}
```

Launch:

```
showIndex<<<3, 4>>>();
```

Complete the table.

Block	Thread	Global Index
0	0	0
0	1	1
0	2	2
0	3	3
1	0	4
1	1	5
1	2	6
1	3	7
2	0	8
2	1	9
2	2	10
2	3	11

Answer:

1. Why is threadIdx.x alone not sufficient for processing a large vector?

threadIdx.x starts from 0 again in every block, so threads in different blocks get the same number and would work on the same element. Also one block can have at most 1024 threads, so threadIdx.x alone can cover only 1024 elements. To give every thread its own element, the block number must also be used

2. What is the value of blockDim.x in this example?

4, because each block has 4 threads (the second value in <<<3, 4>>>)

3. What global index is calculated for thread 2 in block 2?

10, because $2 + 2 \times 4 = 10$

[6] ✓ 0s

```
%%writefile global_index.cu
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void showIndex()
{
    int index =
        threadIdx.x +
        blockIdx.x * blockDim.x;

    printf("Block %d, Thread %d, Global index %d\n",
           blockIdx.x,
           threadIdx.x,
           index);
}

int main()
{
    showIndex<<<3, 4>>>();

    cudaDeviceSynchronize();

    return 0;
}
```

```
Writing global_index.cu
```

[7] ✓ 1s

```
!nvcc global_index.cu -o global_index
!./global_index
```

```
Block 1, Thread 0, Global index 4
Block 1, Thread 1, Global index 5
Block 1, Thread 2, Global index 6
Block 1, Thread 3, Global index 7
Block 0, Thread 0, Global index 0
Block 0, Thread 1, Global index 1
Block 0, Thread 2, Global index 2
Block 0, Thread 3, Global index 3
Block 2, Thread 0, Global index 8
Block 2, Thread 1, Global index 9
Block 2, Thread 2, Global index 10
Block 2, Thread 3, Global index 11
```

Part 4: Host and Device Memory

CUDA programs normally contain separate CPU and GPU memory.

Consider the following variables:

```
int *A;
int *dev_A;
```

Assume that A points to CPU memory and dev A points to GPU memory.

Answer:

1. Which variable refers to host memory?

A. It points to CPU memory

2. Which variable refers to device memory?

dev_A. It points to GPU memory

3. What CUDA function is normally used to allocate memory on the device?

cudaMalloc()

4. What CUDA function is used to copy data between host and device?

cudaMemcpy()

5. What CUDA function is used to release allocated device memory?

cudaFree()

Complete the typical CUDA processing sequence:

1. Allocate host data.
2. Allocate device memory with cudaMalloc()
3. Copy input data from host to device.
4. Launch the kernel on the GPU
5. Copy the result from device to host.
6. Free device memory with cudaFree() and host memory with free()

Part 5: GPU Vector Addition

You will now implement:

$$C_i = A_i + B_i.$$

Use:

$$N = 1,000,000.$$

Each CUDA thread should calculate one element of the result.
Create:

vector_add.cu

Start with:

```
#include <stdio.h>
#include     <stdlib.h>
#include <cuda_runtime.h>

__global__ void vectorAdd(const float *A,
                          const float *B,
                          float *C,
                          int N)
{
    int i =
        threadIdx.x +
        blockIdx.x * blockDim.x;

    if (i < N)
    {
        // TODO:
        // calculate C[i]
    }
}

int main()
{
    const int N = 1000000;

    size_t bytes = N * sizeof(float);

    float *A=(float*)malloc(bytes);
    float *B=(float*)malloc(bytes);
    float *C = (float*)malloc(bytes);

    for (int i = 0; i < N; ++i)
    {
        A[i] = (float)i;
        B[i] = 2.0f * i;
    }

    float *dev_A;
    float *dev_B;
    float *dev_C;

    // TODO:
    // allocate GPU memory

```

```
44
45      // TODO:
46      // copy A and B to GPU memory
47
48
49      int threadsPerBlock = 256;
50
51      int blocks =
52          (N + threadsPerBlock - 1)
53          / threadsPerBlock;
54
55      // TODO:
56      // launch vectorAdd kernel
57
58
59      // Wait until the GPU finishes
60      cudaDeviceSynchronize();
61
62      // TODO:
63      // copy C back to the CPU
64
65
66      printf("C[0] = %f\n", C[0]);
67      printf("C[10] = %f\n", C[10]);
68      printf("C[N-1] = %f\n", C[N-1]);
69
70      // TODO:
71      // free device memory
72
73
74      free(A);
75      free(B);
76      free(C);
77
78      return 0;
79  }
```

Compile:

```
!nvcc vector_add.cu -o vector_add
```

Run:

```
!./vector_add
```

Verify that:

$$C_i = 3i.$$

Therefore:

$$C_{10} = 30.$$

Questions

1. Why is vector addition suitable for GPU execution?

Every element C[i] is calculated separately with the same simple operation, and no element depends on another one. So each thread can take one element, and one million threads can work in parallel

2. Does thread i need the result produced by thread $i - 1$?

No. Thread i only reads A[i] and B[i] and writes C[i]. It does not need the result of any other thread

3. Why do we use:

```
1 if (i < N)
```

inside the kernel?

The number of launched threads is usually a bit larger than N, because every block has a fixed size. Here 3907 blocks × 256 threads = 1000192 threads for 1000000 elements, so 192 threads are extra. Without the check they would read and write outside the arrays

4. Explain the expression:

```
1 (N + threadsPerBlock - 1) / threadsPerBlock
```

It is integer division that rounds up. Plain N / threadsPerBlock rounds down and leaves the last elements without threads. Adding threadsPerBlock - 1 before dividing makes sure the last part also gets a block. For N = 1000000: (1000000 + 255) / 256 = 3907 blocks, while 1000000 / 256 gives only 3906

5. If *N* = 1000 and there are 256 threads per block, how many blocks are required?

4 blocks, because (1000 + 255) / 256 = 4 in integer division. 4 × 256 = 1024 threads, which is enough for 1000 elements, while 3 blocks give only 768

6. Some launched threads may have an index greater than or equal to *N*. What should those threads do?

Nothing. For them the check if (i < N) is false, so they skip the work and finish. In question 5 these are 24 threads (1024 threads for 1000 elements)



```
[8]
✓ 0s
        vectorAdd<<<blocks, threadsPerBlock>>>(dev_A, dev_B, dev_C, N);

        // Wait until the GPU finishes
        cudaDeviceSynchronize();

        // copy C back to the CPU
        cudaMemcpy(C, dev_C, bytes, cudaMemcpyDeviceToHost);

        printf("C[0] = %f\n", C[0]);
        printf("C[10] = %f\n", C[10]);
        printf("C[N-1] = %f\n", C[N - 1]);

        // check every element: C[i] must be 3i
        int errors = 0;
        for (int i = 0; i < N; ++i)
        {
            if (C[i] != 3.0f * i)
            {
                errors++;
            }
        }
        printf("Errors: %d\n", errors);

        // free device memory
        cudaFree(dev_A);
        cudaFree(dev_B);
        cudaFree(dev_C);

        free(A);
        free(B);
        free(C);

        return 0;
    }

••• Writing vector_add.cu

[9]
✓ 1s
    !nvcc vector_add.cu -o vector_add
    !./vector_add

••• C[0] = 0.000000
    C[10] = 30.000000
    C[N-1] = 2999997.000000
    Errors: 0
```

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

__global__ void vectorAdd(const float *A,
                          const float *B,
                          float *C,
                          int N)
{
  int i =
     threadIdx.x +
```

```
        blockIdx.x * blockDim.x;

    if (i < N)
    {
        C[i] = A[i] + B[i];
    }
}

int main()
{
    const int N = 1000000;

    size_t bytes = N * sizeof(float);

    float *A = (float *)malloc(bytes);
    float *B = (float *)malloc(bytes);
    float *C = (float *)malloc(bytes);

    for (int i = 0; i < N; ++i)
    {
        A[i] = (float)i;
        B[i] = 2.0f * i;
    }

    float *dev_A;
    float *dev_B;
    float *dev_C;

    // allocate GPU memory
    cudaMalloc((void **)&dev_A, bytes);
    cudaMalloc((void **)&dev_B, bytes);
    cudaMalloc((void **)&dev_C, bytes);

    // copy A and B to GPU memory
    cudaMemcpy(dev_A, A, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(dev_B, B, bytes, cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;

    int blocks =
        (N + threadsPerBlock - 1)
        / threadsPerBlock;

    // launch vectorAdd kernel
    vectorAdd<<<blocks, threadsPerBlock>>>(dev_A, dev_B, dev_C, N);

    // Wait until the GPU finishes
    cudaDeviceSynchronize();

    // copy C back to the CPU
    cudaMemcpy(C, dev_C, bytes, cudaMemcpyDeviceToHost);

    printf("C[0] = %f\n", C[0]);
    printf("C[10] = %f\n", C[10]);
```

```
    printf("C[N-1] = %f\n", C[N - 1]);

    // check every element: C[i] must be 3i
    int errors = 0;
    for (int i = 0; i < N; ++i)
    {
        if (C[i] != 3.0f * i)
        {
            errors++;
        }
    }
    printf("Errors: %d\n", errors);

    // free device memory
    cudaFree(dev_A);
    cudaFree(dev_B);
    cudaFree(dev_C);

    free(A);
    free(B);
    free(C);

    return 0;
}
```

Part 6: Experiment with Block Size

Run the vector addition program using the following numbers of threads per block:

$$32, \quad 64, \quad 128, \quad 256, \quad 512.$$

For every configuration calculate the required number of blocks.
Use:

$$\text{blocks} = \left\lceil \frac{N}{\text{threads per block}} \right\rceil .$$

Complete the table.

Threads per Block	Number of Blocks
32	31250
64	15625
128	7813
256	3907
512	1954

Answer:

1. Does changing the number of threads per block change the mathematical result?

No. All five runs gave the same C[0], C[10], C[N-1] and 0 errors. Each element is still calculated as A[i] + B[i],

only the way threads are grouped changes

2. Does changing the block size change the way the GPU organizes the computation?

Yes. The same 1000000 elements are split into 31250 small blocks of 32 threads or into 1954 bigger blocks of 512 threads. The GPU sends whole blocks to its processing units (Streaming Multiprocessors), so the number and size of blocks change how the work is spread over the GPU

3. Why do CUDA programs normally use both multiple blocks and multiple threads per block?

One block can have at most 1024 threads and runs on only one Streaming Multiprocessor. The Tesla T4 has 40 of them, so with one block most of the GPU would do nothing, and 1024 threads are not enough for 1000000 elements anyway. Many blocks spread the work over the whole GPU, and many threads in each block keep every Streaming Multiprocessor busy. Threads in one block can also share fast memory and wait for each other

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

__global__ void vectorAdd(const float *A,
                          const float *B,
                          float *C,
                          int N)
{
  int i =
    threadIdx.x +
    blockIdx.x * blockDim.x;

  if (i < N)
  {
    C[i] = A[i] + B[i];
  }
}

int main(int argc, char **argv)
{
  const int N = 1000000;

  size_t bytes = N * sizeof(float);

  float *A = (float *)malloc(bytes);
  float *B = (float *)malloc(bytes);
  float *C = (float *)malloc(bytes);

  for (int i = 0; i < N; ++i)
  {
    A[i] = (float)i;
    B[i] = 2.0f * i;
  }

  float *dev_A;
  float *dev_B;
  float *dev_C;

  cudaMalloc((void **)&dev_A, bytes);
```

```
    cudaMalloc((void **)&dev_B, bytes);
    cudaMalloc((void **)&dev_C, bytes);

    cudaMemcpy(dev_A, A, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(dev_B, B, bytes, cudaMemcpyHostToDevice);

    // threads per block comes from the command line, default is 256
    int threadsPerBlock = 256;
    if (argc > 1)
    {
       threadsPerBlock = atoi(argv[1]);
    }

    int blocks =
       (N + threadsPerBlock - 1)
       / threadsPerBlock;

    vectorAdd<<<blocks, threadsPerBlock>>>(dev_A, dev_B, dev_C, N);

    cudaDeviceSynchronize();

    cudaMemcpy(C, dev_C, bytes, cudaMemcpyDeviceToHost);

    int errors = 0;
    for (int i = 0; i < N; ++i)
    {
       if (C[i] != 3.0f * i)
       {
          errors++;
       }
    }

    printf("Threads per block: %d, Blocks: %d\n", threadsPerBlock, blocks);
    printf("C[0] = %f, C[10] = %f, C[N-1] = %f, Errors: %d\n\n",
          C[0], C[10], C[N - 1], errors);

    cudaFree(dev_A);
    cudaFree(dev_B);
    cudaFree(dev_C);

    free(A);
    free(B);
    free(C);

    return 0;
}
```

[10] ✓ 0s

```
    {
        if (C[i] != 3.0f * i)
        {
            errors++;
        }
    }

    printf("Threads per block: %d, Blocks: %d\n", threadsPerBlock, blocks);
    printf("C[0] = %f, C[10] = %f, C[N-1] = %f, Errors: %d\n\n",
           C[0], C[10], C[N - 1], errors);

    cudaFree(dev_A);
    cudaFree(dev_B);
    cudaFree(dev_C);

    free(A);
    free(B);
    free(C);

    return 0;
}
```

```
Writing vector_add_blocks.cu
```

[11] ✓ 2s

```
!nvcc vector_add_blocks.cu -o vector_add_blocks
!for t in 32 64 128 256 512; do ./vector_add_blocks $t; done
```

```
Threads per block: 32, Blocks: 31250
C[0] = 0.000000, C[10] = 30.000000, C[N-1] = 2999997.000000, Errors: 0

Threads per block: 64, Blocks: 15625
C[0] = 0.000000, C[10] = 30.000000, C[N-1] = 2999997.000000, Errors: 0

Threads per block: 128, Blocks: 7813
C[0] = 0.000000, C[10] = 30.000000, C[N-1] = 2999997.000000, Errors: 0

Threads per block: 256, Blocks: 3907
C[0] = 0.000000, C[10] = 30.000000, C[N-1] = 2999997.000000, Errors: 0

Threads per block: 512, Blocks: 1954
C[0] = 0.000000, C[10] = 30.000000, C[N-1] = 2999997.000000, Errors: 0
```

Part 7: Measuring Kernel Execution Time

CUDA provides event objects that can be used to measure execution time on the GPU.

Add:

```
cudaEvent_t start, stop;

cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord(start);

vectorAdd<<<blocks, threadsPerBlock>>>(
```

```
    dev_A, dev_B, dev_C, N);

cudaEventRecord(stop);

cudaEventSynchronize(stop);

float milliseconds = 0.0f;

cudaEventElapsedTime(
    &milliseconds, start, stop);

printf("Kernel time: %f ms\n",
       milliseconds);

cudaEventDestroy(start);
cudaEventDestroy(stop);
```

Run the experiment several times for each block size.
Complete the table using the average execution time.

Threads per Block	Kernel Time (ms)
32	0.0688
64	0.0520
128	0.0522
256	0.0520
512	0.0517

Answer:

1. Which configuration produced the lowest execution time?

512 threads per block, with an average of 0.0517 ms. But 64, 128 and 256 are almost the same (0.0520 to 0.0522 ms), the difference is less than 1%, which is close to the normal change between runs. Only 32 threads per block is clearly slower, about 1.3 times

2. Is the fastest configuration necessarily the one with the largest number of threads per block?

No. 512 was the fastest, but 64 threads per block gave almost the same time (0.0520 against 0.0517 ms), so more threads per block does not by itself make the kernel faster. Only 32 is clearly slower. On the Tesla T4 one Streaming Multiprocessor can hold at most 16 blocks and 1024 threads at the same time. With 32 threads per block that is only 16 × 32 = 512 threads, half of the maximum, so the GPU has fewer threads to work while others wait for memory. From 64 threads per block (16 × 64 = 1024) the Streaming Multiprocessor can be full, so 64, 128, 256 and 512 give almost the same time

3. Did all configurations produce exactly the same output?

Yes. All five configurations gave C[0] = 0, C[10] = 30, the last element = 2999997 and 0 errors

4. Why should GPU timing experiments normally be repeated several times?

One run can be slowed down by random things, for example other work on the GPU or changes in the GPU clock speed. Here the first timed run was often the slowest, for example 0.0563 ms against about 0.051 ms for 64 threads. The average of several runs gives a more stable and fair number. For the same reason the program also made one warm up launch that was not measured

```
[12]
✓ 0s
            }
        }

        printf("C[0] = %f, C[10] = %f, C[N-1] = %f, Errors: %d\n\n",
               C[0], C[10], C[N - 1], errors);

        cudaFree(dev_A);
        cudaFree(dev_B);
        cudaFree(dev_C);

        free(A);
        free(B);
        free(C);

        return 0;
    }
```

```
Writing vector_add_timing.cu
```

```
[13]
✓ 3s
!nvcc vector_add_timing.cu -o vector_add_timing
!for t in 32 64 128 256 512; do ./vector_add_timing $t; done
```

```
Threads per block: 32
Times (ms): 0.0725 0.0685 0.0696 0.0676 0.0694 0.0693 0.0681 0.0678 0.0676 0.0676
Average kernel time: 0.068806 ms
C[0] = 0.000000, C[10] = 30.000000, C[N-1] = 2999997.000000, Errors: 0

Threads per block: 64
Times (ms): 0.0563 0.0532 0.0512 0.0512 0.0514 0.0512 0.0507 0.0512 0.0525 0.0514
Average kernel time: 0.052038 ms
C[0] = 0.000000, C[10] = 30.000000, C[N-1] = 2999997.000000, Errors: 0

Threads per block: 128
Times (ms): 0.0556 0.0532 0.0516 0.0536 0.0519 0.0514 0.0512 0.0510 0.0512 0.0510
Average kernel time: 0.052186 ms
C[0] = 0.000000, C[10] = 30.000000, C[N-1] = 2999997.000000, Errors: 0

Threads per block: 256
Times (ms): 0.0553 0.0516 0.0512 0.0510 0.0552 0.0512 0.0512 0.0512 0.0512 0.0513
Average kernel time: 0.052042 ms
C[0] = 0.000000, C[10] = 30.000000, C[N-1] = 2999997.000000, Errors: 0

Threads per block: 512
Times (ms): 0.0558 0.0528 0.0520 0.0500 0.0509 0.0512 0.0514 0.0511 0.0512 0.0507
Average kernel time: 0.051722 ms
C[0] = 0.000000, C[10] = 30.000000, C[N-1] = 2999997.000000, Errors: 0
```

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

__global__ void vectorAdd(const float *A,
                          const float *B,
                          float *C,
                          int N)
{
  int i =
    threadIdx.x +
```

```
        blockIdx.x * blockDim.x;

    if (i < N)
    {
        C[i] = A[i] + B[i];
    }
}

int main(int argc, char **argv)
{
    const int N = 1000000;

    size_t bytes = N * sizeof(float);

    float *A = (float *)malloc(bytes);
    float *B = (float *)malloc(bytes);
    float *C = (float *)malloc(bytes);

    for (int i = 0; i < N; ++i)
    {
        A[i] = (float)i;
        B[i] = 2.0f * i;
    }

    float *dev_A;
    float *dev_B;
    float *dev_C;

    cudaMalloc((void **)&dev_A, bytes);
    cudaMalloc((void **)&dev_B, bytes);
    cudaMalloc((void **)&dev_C, bytes);

    cudaMemcpy(dev_A, A, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(dev_B, B, bytes, cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    if (argc > 1)
    {
        threadsPerBlock = atoi(argv[1]);
    }

    int blocks =
        (N + threadsPerBlock - 1)
        / threadsPerBlock;

    // warm up launch, not measured (the first launch is always slower)
    vectorAdd<<<blocks, threadsPerBlock>>>(dev_A, dev_B, dev_C, N);
    cudaDeviceSynchronize();

    cudaEvent_t start, stop;

    cudaEventCreate(&start);
    cudaEventCreate(&stop);
```

```
const int RUNS = 10;
float total = 0.0f;

printf("Threads per block: %d\n", threadsPerBlock);
printf("Times (ms):");

for (int r = 0; r < RUNS; ++r)
{
   cudaEventRecord(start);

   vectorAdd<<<blocks, threadsPerBlock>>>(
      dev_A, dev_B, dev_C, N);

   cudaEventRecord(stop);

   cudaEventSynchronize(stop);

   float milliseconds = 0.0f;

   cudaEventElapsedTime(
      &milliseconds, start, stop);

   printf(" %.4f", milliseconds);
   total += milliseconds;
}

printf("\nAverage kernel time: %f ms\n", total / RUNS);

cudaEventDestroy(start);
cudaEventDestroy(stop);

cudaMemcpy(C, dev_C, bytes, cudaMemcpyDeviceToHost);

int errors = 0;
for (int i = 0; i < N; ++i)
{
   if (C[i] != 3.0f * i)
   {
      errors++;
   }
}

printf("C[0] = %f, C[10] = %f, C[N-1] = %f, Errors: %d\n\n",
     C[0], C[10], C[N - 1], errors);

cudaFree(dev_A);
cudaFree(dev_B);
cudaFree(dev_C);

free(A);
free(B);
free(C);

return 0;
```

}

Part 8: CPU versus GPU

Implement the same operation on the CPU:

```
for (int i = 0; i < N; ++i)
{
    C[i] = A[i] + B[i];
}
```

Use a sufficiently large value of N such that execution time can be measured reliably. For example, try:

$$N = 10^6, \qquad 10^7.$$

Record approximate execution times.

N	CPU Time	GPU Kernel Time
10^6	3.83 ms	0.051 ms
10^7	31.19 ms	0.47 ms

Answer:

1. Which implementation is faster for the larger input?

The GPU kernel. For N = 10^7 it took 0.47 ms against 31.19 ms on the CPU, about 66 times faster. But if copying data is included, the GPU needs 44.52 ms in total and is about 1.4 times slower than the CPU

2. Does the GPU implementation also require time to copy data between CPU and GPU memory?

Yes. For N = 10^7 copying A and B to the GPU took 17.60 ms and copying C back took 26.45 ms. Together that is 44.05 ms, about 99% of the full GPU time of 44.52 ms

3. Why can comparing only kernel execution time with complete CPU execution time be misleading?

The CPU time already includes all the work, but the kernel time shows only the calculation. The GPU also has to receive the input data and send the result back. Here the kernel alone looks 66 times faster, but the full GPU work (44.52 ms) is slower than the CPU (31.19 ms). A fair comparison must include the copies

4. For a very small vector, would you always expect the GPU to be faster? Explain.

No. For a small vector the calculation is tiny, but the GPU still has fixed costs: launching the kernel and copying data between CPU memory and GPU memory. These costs do not become small together with N, so the CPU can finish the whole loop before the GPU even gets the data. Even for N = 10^6 the GPU with copies (4.82 ms) was slower than the CPU (3.83 ms)

```
#include <stdio.h>
#include <stdlib.h>
#include <chrono>
#include <cuda_runtime.h>

__global__ void vectorAdd(const float *A,
                          const float *B,
```

```
                  float *C,
                  int N)
{
   int i =
      threadIdx.x +
      blockIdx.x * blockDim.x;

   if (i < N)
   {
      C[i] = A[i] + B[i];
   }
}

// the same operation on the CPU
void cpuAdd(const float *A, const float *B, float *C, int N)
{
   for (int i = 0; i < N; ++i)
   {
      C[i] = A[i] + B[i];
   }
}

int main(int argc, char **argv)
{
   // N comes from the command line, default is 1000000
   int N = 1000000;
   if (argc > 1)
   {
      N = atoi(argv[1]);
   }

   size_t bytes = (size_t)N * sizeof(float);

   float *A = (float *)malloc(bytes);
   float *B = (float *)malloc(bytes);
   float *C_cpu = (float *)malloc(bytes);
   float *C_gpu = (float *)malloc(bytes);

   for (int i = 0; i < N; ++i)
   {
      A[i] = (float)i;
      B[i] = 2.0f * i;
   }

   const int RUNS = 10;

   // CPU: one warm up run, then the average of 10 runs
   cpuAdd(A, B, C_cpu, N);

   double cpuTotal = 0.0;
   for (int r = 0; r < RUNS; ++r)
   {
      auto t0 = std::chrono::high_resolution_clock::now();
      cpuAdd(A, B, C_cpu, N);
```

```
    auto t1 = std::chrono::high_resolution_clock::now();
    cpuTotal += std::chrono::duration<double, std::milli>(t1 - t0).count();
}
double cpuTime = cpuTotal / RUNS;

// GPU
float *dev_A;
float *dev_B;
float *dev_C;

cudaMalloc((void **)&dev_A, bytes);
cudaMalloc((void **)&dev_B, bytes);
cudaMalloc((void **)&dev_C, bytes);

cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);

// time of copying A and B from host to device
cudaEventRecord(start);
cudaMemcpy(dev_A, A, bytes, cudaMemcpyHostToDevice);
cudaMemcpy(dev_B, B, bytes, cudaMemcpyHostToDevice);
cudaEventRecord(stop);
cudaEventSynchronize(stop);
float h2dTime = 0.0f;
cudaEventElapsedTime(&h2dTime, start, stop);

int threadsPerBlock = 256;
int blocks =
    (N + threadsPerBlock - 1)
    / threadsPerBlock;

// kernel: one warm up launch, then the average of 10 launches
vectorAdd<<<blocks, threadsPerBlock>>>(dev_A, dev_B, dev_C, N);
cudaDeviceSynchronize();

float kernelTotal = 0.0f;
for (int r = 0; r < RUNS; ++r)
{
    cudaEventRecord(start);
    vectorAdd<<<blocks, threadsPerBlock>>>(dev_A, dev_B, dev_C, N);
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);

    float milliseconds = 0.0f;
    cudaEventElapsedTime(&milliseconds, start, stop);
    kernelTotal += milliseconds;
}
float kernelTime = kernelTotal / RUNS;

// time of copying C from device to host
cudaEventRecord(start);
cudaMemcpy(C_gpu, dev_C, bytes, cudaMemcpyDeviceToHost);
cudaEventRecord(stop);
```

```
    cudaEventSynchronize(stop);
    float d2hTime = 0.0f;
    cudaEventElapsedTime(&d2hTime, start, stop);

    // the GPU result must be the same as the CPU result
    int errors = 0;
    for (int i = 0; i < N; ++i)
    {
        if (C_gpu[i] != C_cpu[i])
        {
            errors++;
        }
    }

    printf("N = %d\n", N);
    printf("CPU time: %f ms\n", cpuTime);
    printf("GPU kernel time: %f ms\n", kernelTime);
    printf("Copy host to device: %f ms\n", h2dTime);
    printf("Copy device to host: %f ms\n", d2hTime);
    printf("GPU total (copies + kernel): %f ms\n",
          h2dTime + kernelTime + d2hTime);
    printf("Errors: %d\n\n", errors);

    cudaEventDestroy(start);
    cudaEventDestroy(stop);

    cudaFree(dev_A);
    cudaFree(dev_B);
    cudaFree(dev_C);

    free(A);
    free(B);
    free(C_cpu);
    free(C_gpu);

    return 0;
}
```

[14] ✓ 0s

```
        printf("N = %d\n", N);
        printf("CPU time: %f ms\n", cpuTime);
        printf("GPU kernel time: %f ms\n", kernelTime);
        printf("Copy host to device: %f ms\n", h2dTime);
        printf("Copy device to host: %f ms\n", d2hTime);
        printf("GPU total (copies + kernel): %f ms\n",
               h2dTime + kernelTime + d2hTime);
        printf("Errors: %d\n\n", errors);

        cudaEventDestroy(start);
        cudaEventDestroy(stop);

        cudaFree(dev_A);
        cudaFree(dev_B);
        cudaFree(dev_C);

        free(A);
        free(B);
        free(C_cpu);
        free(C_gpu);

        return 0;
    }
```

```
Writing cpu_vs_gpu.cu
```

[15] ✓ 2s

```
!nvcc cpu_vs_gpu.cu -o cpu_vs_gpu
!./cpu_vs_gpu 1000000
!./cpu_vs_gpu 10000000
```

```
N = 1000000
CPU time: 3.826538 ms
GPU kernel time: 0.050867 ms
Copy host to device: 2.019168 ms
Copy device to host: 2.749760 ms
GPU total (copies + kernel): 4.819795 ms
Errors: 0

N = 10000000
CPU time: 31.186281 ms
GPU kernel time: 0.471658 ms
Copy host to device: 17.601376 ms
Copy device to host: 26.451040 ms
GPU total (copies + kernel): 44.524071 ms
Errors: 0
```

Part 9: Two-Dimensional Thread Indexing

GPU programs are not limited to one-dimensional data.

Consider a matrix:

$$A \in R^{N \times N}.$$

For a two-dimensional block, a thread can determine its matrix position using:

```
int col =
    blockIdx.x * blockDim.x +
    threadIdx.x;

int row =
    blockIdx.y * blockDim.y +
    threadIdx.y;
```

Suppose:

$$N = 512.$$

Use:

```
dim3 threads(16, 16);

dim3 blocks(
    (N + threads.x - 1) / threads.x,
    (N + threads.y - 1) / threads.y);
```

Answer:

1. How many threads are contained in one 16 × 16 block?

256 threads, because 16 × 16 = 256

2. How many blocks are needed in the *x* direction for *N* = 512?

32 blocks, because (512 + 15) / 16 = 32 in integer division. 32 × 16 = 512, exactly one thread for each column

3. How many blocks are needed in the *y* direction?

32 blocks, the same calculation as for x, because the matrix is square: (512 + 15) / 16 = 32

4. How many blocks exist in the complete grid?

1024 blocks, because 32 × 32 = 1024. Together they have 1024 × 256 = 262144 threads, which is exactly 512 × 512, one thread for each matrix element

5. Why is two-dimensional thread indexing convenient for matrices and images?

A matrix or an image has rows and columns. With 2D indexing each thread gets its column from the x values and its row from the y values, so its position in the grid matches the position of its element. The thread does not need extra math to turn one number into a row and a column

Part 10: Matrix Addition

Implement:

$$C_{i,j} = A_{i,j} + B_{i,j}.$$

Use one GPU thread for each matrix element.
Complete the kernel:

```
__global__ void matrixAdd(
    const float *A,
    const float *B,
    float *C,
    int width,
    int height)
{
    int col =
        blockIdx.x * blockDim.x +
        threadIdx.x;

    int row =
        blockIdx.y * blockDim.y +
        threadIdx.y;

    if (row < height && col < width)
    {
        int index = row * width + col;

        // TODO:
        // calculate C[index]
    }
}
```

Use:

$$512 \times 512$$

matrices.
Launch the kernel using:

```
dim3 threads(16, 16);

dim3 blocks(
    (width + threads.x - 1) / threads.x,
    (height + threads.y - 1) / threads.y);

matrixAdd<<<blocks, threads>>>(
    dev_A,
    dev_B,
    dev_C,
    width,
    height);
```

Verify several elements of the resulting matrix.

Questions

1. Which thread calculates one matrix element?

One thread for each element. Its column is blockIdx.x × blockDim.x + threadIdx.x and its row is blockIdx.y × blockDim.y + threadIdx.y. For example, C[100][200] is calculated by the thread with blockIdx = (12, 6) and

threadIdx = (8, 4), because 12 × 16 + 8 = 200 and 6 × 16 + 4 = 100

2. Why are both row and column indices required?

One element of a matrix is defined by two numbers. The row alone points to a whole row of 512 elements, and the column alone points to a whole column. Both are needed to find one element and to check both borders (row < height and col < width)

3. Why is the matrix stored in a one-dimensional array in this example?

GPU memory is one long line of addresses, and cudaMalloc and cudaMemcpy work with one continuous piece of memory. So the matrix is stored row after row in one array. This needs one allocation and one copy for the whole matrix instead of 512 separate rows

4. What is the purpose of:

```
row * width + col
```

?

It turns the position (row, col) into the position in the 1D array: skip row full rows of width elements, then move col more elements. For example, C[1][0] is at 1 × 512 + 0 = 512, so its value is 3 × 512 = 1536, the same as in the output

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

__global__ void matrixAdd(
   const float *A,
   const float *B,
   float *C,
   int width,
   int height)
{
   int col =
      blockIdx.x * blockDim.x +
      threadIdx.x;

   int row =
      blockIdx.y * blockDim.y +
      threadIdx.y;

   if (row < height && col < width)
   {
      int index = row * width + col;

      C[index] = A[index] + B[index];
   }
}

int main()
{
   const int width = 512;
   const int height = 512;
   const int total = width * height;

   size_t bytes = total * sizeof(float);
```

```
float *A = (float *)malloc(bytes);
float *B = (float *)malloc(bytes);
float *C = (float *)malloc(bytes);

for (int i = 0; i < total; ++i)
{
   A[i] = (float)i;
   B[i] = 2.0f * i;
}

float *dev_A;
float *dev_B;
float *dev_C;

cudaMalloc((void **)&dev_A, bytes);
cudaMalloc((void **)&dev_B, bytes);
cudaMalloc((void **)&dev_C, bytes);

cudaMemcpy(dev_A, A, bytes, cudaMemcpyHostToDevice);
cudaMemcpy(dev_B, B, bytes, cudaMemcpyHostToDevice);

dim3 threads(16, 16);

dim3 blocks(
   (width + threads.x - 1) / threads.x,
   (height + threads.y - 1) / threads.y);

matrixAdd<<<blocks, threads>>>(
   dev_A,
   dev_B,
   dev_C,
   width,
   height);

cudaDeviceSynchronize();

cudaMemcpy(C, dev_C, bytes, cudaMemcpyDeviceToHost);

printf("Grid: %d x %d blocks, block: %d x %d threads\n",
      blocks.x, blocks.y, threads.x, threads.y);

// print several elements: (row, col)
int rows[] = {0, 0, 1, 100, 511};
int cols[] = {0, 1, 0, 200, 511};

for (int k = 0; k < 5; ++k)
{
   int index = rows[k] * width + cols[k];
   printf("C[%d][%d] = %f (expected %f)\n",
         rows[k], cols[k], C[index], 3.0f * index);
}

// check every element: C[index] must be 3 * index
```

```
    int errors = 0;
    for (int i = 0; i < total; ++i)
    {
        if (C[i] != 3.0f * i)
        {
            errors++;
        }
    }
    printf("Errors: %d\n", errors);

    cudaFree(dev_A);
    cudaFree(dev_B);
    cudaFree(dev_C);

    free(A);
    free(B);
    free(C);

    return 0;
}
```

[16] ✓ 0s

```
        // print several elements: (row, col)
        int rows[] = {0, 0, 1, 100, 511};
        int cols[] = {0, 1, 0, 200, 511};

        for (int k = 0; k < 5; ++k)
        {
            int index = rows[k] * width + cols[k];
            printf("C[%d][%d] = %f (expected %f)\n",
                   rows[k], cols[k], C[index], 3.0f * index);
        }

        // check every element: C[index] must be 3 * index
        int errors = 0;
        for (int i = 0; i < total; ++i)
        {
            if (C[i] != 3.0f * i)
            {
                errors++;
            }
        }
        printf("Errors: %d\n", errors);

        cudaFree(dev_A);
        cudaFree(dev_B);
        cudaFree(dev_C);

        free(A);
        free(B);
        free(C);

        return 0;
    }
```

```
Writing matrix_add.cu
```

[17] ✓ 1s

```
!nvcc matrix_add.cu -o matrix_add
!./matrix_add
```

```
Grid: 32 x 32 blocks, block: 16 x 16 threads
C[0][0] = 0.000000 (expected 0.000000)
C[0][1] = 3.000000 (expected 3.000000)
C[1][0] = 1536.000000 (expected 1536.000000)
C[100][200] = 154200.000000 (expected 154200.000000)
C[511][511] = 786429.000000 (expected 786429.000000)
Errors: 0
```

Part 11: CUDA Memory Model

For each type of memory, indicate its scope.

Memory	Visible To
Registers	One thread only
Local memory	One thread only
Shared memory	All threads of one block
Global memory	All threads of all blocks in the grid, and the host through cudaMemcpy()

Answer:

1. Which memory is visible to all threads in a grid?

Global memory

2. Which memory is shared by threads belonging to the same block?

Shared memory

3. Can threads in different blocks directly use the same shared-memory array?

No. Every block has its own copy of a shared memory array, which is stored in its Streaming Multiprocessor. Blocks can exchange data only through global memory

4. Why can shared memory be faster than global memory?

Shared memory is located on the GPU chip, inside the Streaming Multiprocessor, close to the threads. Global memory is the separate GPU memory outside the chip (the 15 GB on the Tesla T4), so every access to it takes much more time

5. Why is data reuse important when optimizing GPU programs?

Reading from global memory is slow, so many kernels spend more time waiting for data than calculating. If the same data is needed many times, it can be loaded once into shared memory or registers and then reused, so the kernel makes fewer slow global reads. For example, in matrix multiplication every element of A is used N times. In our vector addition there is no reuse, every element is read only once, so its speed is limited by global memory

Part 12: Synchronization

Consider several threads belonging to one thread block.

CUDA provides:

```
1 __syncthreads();
```

Answer:

1. What is the purpose of __syncthreads()?

It is a barrier inside a block. Every thread of the block stops at __syncthreads() and waits until all other threads of the same block reach it. Only then all of them continue

2. Does it synchronize threads belonging to one block or the entire grid?

Only threads of one block. Threads in other blocks do not wait. To wait for the whole grid, the host must wait until the kernel finishes, for example with cudaDeviceSynchronize()

3. Why might synchronization be required after several threads write data to shared memory?

Threads run in parallel and in no fixed order. If a thread reads a shared memory value that another thread has not written yet, it gets an old or random value. __syncthreads() after the writes makes sure all values are written before any thread starts reading them

4. Can we assume that CUDA threads finish in numerical order?

No. In Part 2 and Part 3 the messages were not printed in a fixed order, and threads can finish in any order

5. Can we assume that CUDA blocks finish in numerical order?

No. In Part 3 the blocks were printed in the order 1, 0, 2. The GPU runs blocks in any order it chooses

Part 13: Short Discussion

Answer each question briefly.

1. What is the difference between the host and the device?

The host is the CPU and its memory, the device is the GPU and its memory. The host runs main(), prepares data and launches kernels, the device runs the kernels

2. What is a CUDA kernel?

A function that runs on the GPU, is marked with __global__ and is launched from the host with <<<blocks, threads>>>. Many threads run it in parallel

3. What is the difference between a thread, a block, and a grid?

A thread runs the kernel once for one piece of data. A block is a group of threads that run on one Streaming Multiprocessor and can share memory. A grid is all blocks of one kernel launch

4. What is the purpose of threadIdx?

It gives the number of the thread inside its block

5. What is the purpose of blockIdx?

It gives the number of the block inside the grid

6. What is the purpose of blockDim?

It gives the number of threads in one block. Together with blockIdx and threadIdx it is used to calculate the global index

7. Why is GPU computing particularly suitable for data-parallel problems?

In such problems the same operation is done on many elements independently, like C[i] = A[i] + B[i]. A GPU has thousands of simple cores, so it can process thousands of elements at the same time

8. What is the purpose of cudaMalloc()?

It allocates memory on the GPU

9. What is the purpose of cudaMemcpy()?

It copies data between host memory and device memory, in either direction

10. Why must host-to-device and device-to-host transfers be considered when evaluating GPU performance?

The GPU can work only with data in its own memory, so the input must be copied to it and the result copied back. In Part 8 for N = 10^7 the kernel took 0.47 ms but the copies took 44.05 ms, so without them the GPU looks much faster than it really is

11. Why are thousands of CUDA threads commonly used instead of only a few threads?

Each thread does only a small piece of work, and while some threads wait for memory, the GPU switches to others. With only a few threads most of the GPU cores would do nothing. In Part 7, 32 threads per block could fill only half of each Streaming Multiprocessor and the kernel was about 1.3 times slower

12. Why can several different CUDA execution configurations produce the same correct result but different performance?

The result depends only on what each thread calculates, not on how threads are grouped, so every configuration gives the same answer. But the grouping changes how well the GPU is filled with work. In Part 6 and Part 7 all block sizes gave 0 errors, but 32 threads per block took 0.0688 ms while 512 took 0.0517 ms

Optional Challenge: Matrix Multiplication

This part is optional.

Implement the matrix multiplication:

$$C = A \times B.$$

Each GPU thread should calculate one element:

$$C_{i,j} = \sum_{k=0}^{N-1} A_{i,k} B_{k,j}.$$

Use a two-dimensional grid and two-dimensional blocks.

You do **not** need to use shared-memory tiling for this exercise.

Test your implementation using small matrices first, and compare the GPU result against a CPU implementation.

```
#include <stdio.h>
#include <stdlib.h>
#include <chrono>
#include <cuda_runtime.h>

// each thread calculates one element C[row][col]
__global__ void matrixMul(const float *A,
                          const float *B,
                          float *C,
                          int N)
{
  int col =
    blockIdx.x * blockDim.x +
    threadIdx.x;

  int row =
    blockIdx.y * blockDim.y +
    threadIdx.y;

  if (row < N && col < N)
  {
    float sum = 0.0f;

    for (int k = 0; k < N; ++k)
    {
      sum += A[row * N + k] * B[k * N + col];
    }

    C[row * N + col] = sum;
  }
}

// the same calculation on the CPU
void cpuMatrixMul(const float *A, const float *B, float *C, int N)
{
  for (int row = 0; row < N; ++row)
  {
```

```
      for (int col = 0; col < N; ++col)
      {
         float sum = 0.0f;

         for (int k = 0; k < N; ++k)
         {
            sum += A[row * N + k] * B[k * N + col];
         }

         C[row * N + col] = sum;
      }
   }
}

void printMatrix(const char *name, const float *M, int N)
{
   printf("%s:\n", name);
   for (int row = 0; row < N; ++row)
   {
      for (int col = 0; col < N; ++col)
      {
         printf("%6.0f", M[row * N + col]);
      }
      printf("\n");
   }
}

int main(int argc, char **argv)
{
   // N comes from the command line, default is 4
   int N = 4;
   if (argc > 1)
   {
      N = atoi(argv[1]);
   }

   size_t bytes = (size_t)N * N * sizeof(float);

   float *A = (float *)malloc(bytes);
   float *B = (float *)malloc(bytes);
   float *C_cpu = (float *)malloc(bytes);
   float *C_gpu = (float *)malloc(bytes);

   // small whole numbers, so CPU and GPU results must be exactly equal
   for (int i = 0; i < N * N; ++i)
   {
      A[i] = (float)(i % 10);
      B[i] = (float)(i % 7);
   }

   float *dev_A;
   float *dev_B;
   float *dev_C;
```

```
cudaMalloc((void **)&dev_A, bytes);
cudaMalloc((void **)&dev_B, bytes);
cudaMalloc((void **)&dev_C, bytes);

cudaMemcpy(dev_A, A, bytes, cudaMemcpyHostToDevice);
cudaMemcpy(dev_B, B, bytes, cudaMemcpyHostToDevice);

dim3 threads(16, 16);

dim3 blocks(
   (N + threads.x - 1) / threads.x,
   (N + threads.y - 1) / threads.y);

cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord(start);

matrixMul<<<blocks, threads>>>(dev_A, dev_B, dev_C, N);

cudaEventRecord(stop);
cudaEventSynchronize(stop);

float gpuTime = 0.0f;
cudaEventElapsedTime(&gpuTime, start, stop);

cudaMemcpy(C_gpu, dev_C, bytes, cudaMemcpyDeviceToHost);

auto t0 = std::chrono::high_resolution_clock::now();
cpuMatrixMul(A, B, C_cpu, N);
auto t1 = std::chrono::high_resolution_clock::now();
double cpuTime =
   std::chrono::duration<double, std::milli>(t1 - t0).count();

// compare GPU and CPU results element by element
int errors = 0;
for (int i = 0; i < N * N; ++i)
{
   if (C_gpu[i] != C_cpu[i])
   {
      errors++;
   }
}

printf("N = %d, grid: %d x %d blocks\n", N, blocks.x, blocks.y);

if (N <= 8)
{
   printMatrix("A", A, N);
   printMatrix("B", B, N);
   printMatrix("C (GPU)", C_gpu, N);
```

```
    }

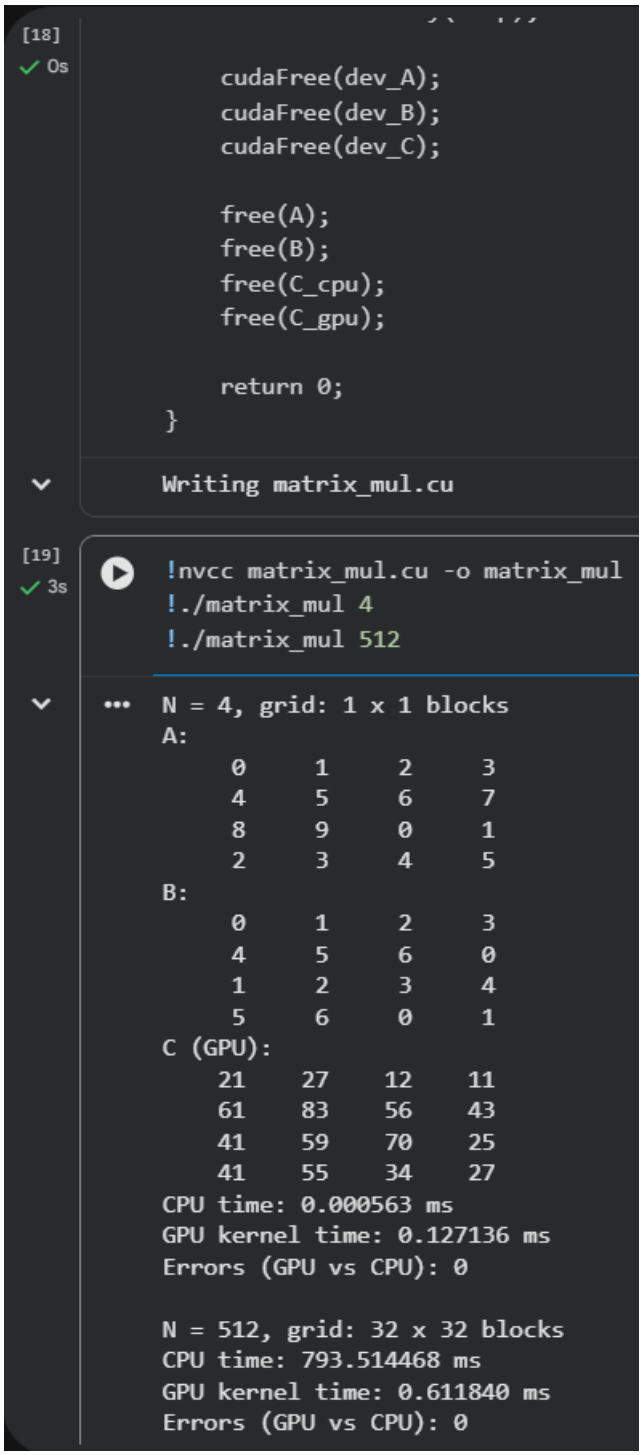
    printf("CPU time: %f ms\n", cpuTime);
    printf("GPU kernel time: %f ms\n", gpuTime);
    printf("Errors (GPU vs CPU): %d\n\n", errors);

    cudaEventDestroy(start);
    cudaEventDestroy(stop);

    cudaFree(dev_A);
    cudaFree(dev_B);
    cudaFree(dev_C);

    free(A);
    free(B);
    free(C_cpu);
    free(C_gpu);

    return 0;
}
```



Each GPU thread calculates one element C[row][col] as the sum of A[row][k] × B[k][col] over all k. The kernel uses a 2D grid of 16 × 16 blocks. The GPU result was compared with a CPU implementation of the same calculation.

For N = 4 the result can be checked by hand, for example C[0][0] = 0 × 0 + 1 × 4 + 2 × 1 + 3 × 5 = 21. For both N = 4 and N = 512 there were 0 errors between GPU and CPU.

For N = 512 the CPU needed 793.51 ms and the GPU kernel only 0.61 ms, about 1300 times faster. For N = 4 the CPU was faster (0.0006 ms against 0.13 ms), because the work is tiny and the GPU has a fixed cost for every launch. The times include only the calculation, not the copies.

Matrix multiplication gains much more from the GPU than vector addition. In vector addition each element needs one addition, but here each element needs 512 multiplications and additions, so there is much more

calculation for the same amount of data.

Submission

Submit:

1. your Google Colab notebook (.ipynb);
2. a screenshot or saved output of nvidia-smi showing the GPU used;
3. hello.cu;
4. the completed vector addition program;
5. the completed matrix addition program;
6. the block-size performance table;
7. the CPU/GPU comparison table;
8. answers to all questions.

The submitted Colab notebook should contain the program code and its execution output. All CUDA programs must compile successfully using nvcc.

Conclusion

In this lab I wrote and ran CUDA programs on a Tesla T4 GPU in Google Colab. I learned how the host launches kernels on the device, how threads are grouped into blocks and a grid, and how each thread finds its own element with the global index. Vector addition and matrix addition gave correct results with 0 errors, and the result did not depend on the block size.

The experiments showed that the GPU kernel is much faster than the CPU: for N = 10^7 vector addition took 0.47 ms on the GPU and 31.19 ms on the CPU, and 512 × 512 matrix multiplication was about 1300 times faster. But copying data between CPU and GPU memory took 44.05 ms, so for vector addition the full GPU work was slower than the CPU. The GPU pays off when there is a lot of calculation for each piece of data, and a fair comparison must include the copies. Block size matters less than the total number of threads: only 32 threads per block was clearly slower.